

Question 1 answer: The finally block ensures code inside it runs regardless of whether an exception is thrown, typically for cleanup.

Question 2 answer: `int.TryParse()` avoids exceptions by returning false on invalid input instead of throwing, improving stability.

Question 3 answer: `InvalidOperationException` occurs when accessing Value on a null `Nullable<T>`.

Question 4 answer: Checking bounds prevents `IndexOutOfRangeException` and avoids accessing invalid memory.

Question 5 answer: `GetLength(dimension)` returns the size of a specified dimension in a multi-dimensional array.

Question 6 answer: Rectangular arrays allocate one contiguous block of memory; jagged arrays allocate separate arrays for each row.

Question 7 answer: Nullable reference types help catch possible null usage at compile time to reduce `NullReferenceExceptions`.

Question 8 answer: Boxing/unboxing incurs extra memory and CPU cost because value types are wrapped/unwrapped into objects.

Question 9 answer: out parameters must be assigned inside the method to ensure they have a definite value when returned.

Question 10 answer: Optional parameters must be last so callers can omit them without ambiguity in argument order.

Question 11 answer: The null propagation operator (`?.`) stops evaluation if the left side is null, preventing `NullReferenceException`.

Question 12 answer: A switch expression is preferred when mapping input to output concisely, especially for pattern matching.

Question 13 answer: params must be the last parameter, can only have one per method, and works only with a single array type.